

## • Functional Requirements

1. User submits Long URL and gets Short URL
2. System generates a Short URL
3. When someone clicks the Short URL, it redirects to the original URL.

## • Non Functional Requirements

1. High availability
2. Low Latency
3. Highly scalable
4. Fault tolerant

## • High Level Architecture

Client

|

API Gateway

|

URL Shortening Service

|

Database

|

Cache (Redis)

## • Database Design

MySQL

Primary key → id



Table:

| Id | Long URL | Short URL | created at |
|----|----------|-----------|------------|
|----|----------|-----------|------------|

- Hashing

Hash the URL

hash (URL)  $\rightarrow$  short URL

- Random ID

generate random ID

- Caching

To speed up redirection, use cache like Redis

- Scaling the System

1. Load Balancer like NGINX
2. Scale databases using the techniques such as Replication and Sharding.

Final Architecture

User



DNS



Load Balancer



Application Server



Cache (Redis) — Database